SY B.Tech. – ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING PRACTICAL ASSIGNMENT

1. Student Details

Student Name	Pratik Akrur PAndit
PRN	202501110116
Division	B
Batch	B2
Roll No.	202501110116
Date of Submission	26/10/2026

2. Practical Details

Practical No.	7
Practical Title	Polymorphism
Date Performed	21/10/2026
Faculty Name	Shruti Gadewar

3. Problem Statement

Write a C++ program that demonstrates polymorphism. The program must show compile-time (static) polymorphism using function overloading and operator overloading, and runtime (dynamic) polymorphism using function overriding and virtual functions through base-class pointers.

4. Objectives

1. To understand the concept of polymorphism ("many forms") in Object Oriented Programming.
2. To implement compile-time polymorphism using function overloading and operator overloading.
3. To implement runtime polymorphism using inheritance, function overriding and virtual functions.

5. Expected Learning Outcomes

After completing this practical, the student will be able to:

1. Differentiate between early binding (compile-time) and late binding (runtime) polymorphism.
2. Overload functions and operators to give the same name or symbol different behaviours.
3. Use virtual functions and base-class pointers so that the function called depends on the type of the object, not the type of the pointer.

6. Theory / Concept

Polymorphism means having many forms. In C++, it typically occurs when classes are related by inheritance, and a call to a member function executes a different function depending on the type of object that invokes it. Polymorphism is of two types:

A. Compile-time (Static / Early Binding) Polymorphism

- Resolved by the compiler before the program runs.
- *Function overloading*: several functions share the same name but differ in the number, type or order of parameters.

- *Operator overloading*: operators such as + are given a new meaning for user-defined types (for example, adding two complex numbers).

B. Runtime (Dynamic / Late Binding) Polymorphism

- Also called dynamic method dispatch; the call is resolved during program execution, not by the compiler.
- *Function overriding*: a derived class redefines a method already present in the base class.
- *Virtual function*: a base-class function declared with the keyword **virtual**. It tells the compiler not to use static linkage; instead, the function to call is selected based on the kind of object for which it is called (dynamic linkage / late binding). It is achieved through base-class pointers.
- Runtime polymorphism is slightly slower than static polymorphism but more flexible.

Advantages of polymorphism: it allows programs to be extended easily, lets tested classes and code be reused, and reduces coupling between different functions.

7. Algorithm / Pseudocode

```
START
Step 1: Define class Calculator with three overloaded add() functions
        (int,int), (double,double) and (int,int,int).
Step 2: Define class Complex and overload operator+ to add two complex numbers.
Step 3: Define base class Shape with virtual functions area() and show(), and
        a virtual destructor.
Step 4: Define derived classes Circle, Rectangle and Triangle that override
        area() and show().
Step 5: In main(), call the overloaded add() functions and add two Complex objects
        using the + operator (compile-time polymorphism).
Step 6: Create an array of Shape* pointing to Shape, Circle, Rectangle and
        Triangle objects; call show() through each pointer (runtime polymorphism).
Step 7: Delete all dynamically created objects and display the output.
STOP
```

08. Technology / Tools / Libraries Used

Technology / Tool	Purpose
Programming Language	C++ (C++11 standard)
IDE / Platform	Terminal / text editor on Linux
Compiler	g++ (GCC)
Libraries / Packages	<iostream> for input/output (standard library only)

09. Implementation / GitHub Repository

GitHub Repository URL	https://github.com/panditpratik/OOP-
Repository Name	**Problem-Solving-Using-OOP-in-Cpp**
Branch Used	main
Commit ID / Latest Commit	7f8fc4031843bc9245fef14b740b25841f3d265b
Date of Last Commit	24/10/26

10. Test Cases / Input and Output

Test Case	Input	Expected Output	Actual Output	Status
TC1 add(int, int)	10, 20	30	30	Pass
TC2 add(double, double)	2.5, 3.5	6	6	Pass
TC3 add(int, int, int)	1, 2, 3	6	6	Pass
TC4 operator + (Complex)	(3 + 4i), (1 + 2i)	4 + 6i	4 + 6i	Pass
TC5 Shape* to Shape	-	Generic Shape, Area = 0	Generic Shape, Area = 0	Pass
TC6 Shape* to Circle	r = 5	Circle (r = 5), Area = 78.5397	Circle (r = 5), Area = 78.5397	Pass
TC7 Shape* to Rectangle	4, 6	Rectangle (4 x 6), Area = 24	Rectangle (4 x 6), Area = 24	Pass
TC8 Shape* to Triangle	b = 3, h = 8	Triangle (b = 3, h = 8), Area = 12	Triangle (b = 3, h = 8), Area = 12	Pass

11. Results / Observations

- The compiler selected the correct add() function at compile time, based on the number and type of arguments (function overloading).
- The + operator worked on Complex objects exactly as it does on built-in types (operator overloading).
- Although every element of the array is of type Shape*, each call to show() executed the version belonging to the actual object (Shape, Circle, Rectangle or Triangle). This shows late binding through virtual functions.
- Declaring the destructor virtual ensured that derived-class destructors run correctly when objects are deleted through base-class pointers.

Screenshots / Graphs / Output:

Pratical_assignment_7.cpp > main()

```cpp
#include <iostream>
using namespace std;

// Compile-time polymorphism: Function Overloading
class Calculator {
public:
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
};

// Compile-time polymorphism: Operator Overloading
class Complex {
    double real, imag;

public:
    Complex(double r = 0, double i = 0) : real(r), imag(i) {}

    Complex operator+(const Complex &other) const {
        return Complex(real + other.real, imag + other.imag);
    }

    void display() const {
        cout << real << " + " << imag << "i" << endl;
    }
};

// Runtime polymorphism: Virtual Functions and Overriding
class Shape {
public:
    virtual double area() const {
        return 0;
    }

    virtual void show() const {
        cout << "Generic Shape, Area = " << area() << endl;
    }

    virtual ~Shape() {}
```

```
47      virtual ~Shape() {}
48  };
49
50  class Circle : public Shape {
51      double r;
52
53  public:
54      Circle(double radius) : r(radius) {}
55
56      double area() const override {
57          return 3.14159 * r * r;
58      }
59
60      void show() const override {
61          cout << "Circle (r = " << r << "), Area = " << area() << endl;
62      }
63  };
64
65  class Rectangle : public Shape {
66      double l, b;
67
68  public:
69      Rectangle(double length, double breadth) : l(length), b(breadth) {}
70
71      double area() const override {
72          return l * b;
73      }
74
75      void show() const override {
76          cout << "Rectangle (" << l << " x " << b
77               << "), Area = " << area() << endl;
78      }
79  };
80
81  class Triangle : public Shape {
82      double base, height;
83
84  public:
85      Triangle(double b, double h) : base(b), height(h) {}
86
87      double area() const override {
88          return 0.5 * base * height;
89      }
90
91      void show() const override {
```

```
91      void show() const override {
92          cout << "Triangle (b = " << base << ", h = " << height
93               << "), Area = " << area() << endl;
94      }
95  };
96
97  int main() {
98      cout << "===== Compile-time Polymorphism =====" << endl;
99
100     cout << "-- Function Overloading --" << endl;
101     Calculator calc;
102
103     cout << "add(10, 20) = " << calc.add(10, 20) << endl;
104     cout << "add(2.5, 3.5) = " << calc.add(2.5, 3.5) << endl;
105     cout << "add(1, 2, 3) = " << calc.add(1, 2, 3) << endl;
106
107     cout << "-- Operator Overloading --" << endl;
108     Complex c1(3, 4), c2(1, 2), c3;
109
110     cout << "c1 = ";
111     c1.display();
112
113     cout << "c2 = ";
114     c2.display();
115
116     c3 = c1 + c2;
117
118     cout << "c1 + c2 = ";
119     c3.display();
120
121     cout << endl << "===== Runtime Polymorphism =====" << endl;
122
123     Shape *shapes[4] = {
124         new Shape(),
125         new Circle(5),
126         new Rectangle(4, 6),
127         new Triangle(3, 8)
128     };
129
130     for (int i = 0; i < 4; i++) {
131         shapes[i]->show();
132     }
133
134     for (int i = 0; i < 4; i++) {
135         delete shapes[i];
```

- Output:-

```
PS D:\All Programs\OOP(C++)> cd "d:\All Programs\OOP(C++)\" ;
===== Compile-time Polymorphism =====
-- Function Overloading --
add(10, 20) = 30
add(2.5, 3.5) = 6
add(1, 2, 3) = 6
-- Operator Overloading --
c1 = 3 + 4i
c2 = 1 + 2i
c1 + c2 = 4 + 6i

===== Runtime Polymorphism =====
Generic Shape, Area = 0
Circle (r = 5), Area = 78.5397
Rectangle (4 x 6), Area = 24
Triangle (b = 3, h = 8), Area = 12
PS D:\All Programs\OOP(C++)>
```

12. Conclusion

In this practical, a C++ program was implemented to demonstrate both types of polymorphism. Compile-time polymorphism was achieved using function overloading and operator overloading, while runtime polymorphism was achieved using inheritance, function overriding and virtual functions with base-class pointers. I learned that polymorphism makes code more flexible, reusable and easier to extend, since new derived classes can be added without changing the existing code that uses the base class.

13. Assignment Questions

Q1. Differentiate between compile-time and runtime polymorphism.

Answer:

Basis	Compile-time Polymorphism	Runtime Polymorphism
Also called	Static / early binding	Dynamic / late binding
Resolved by	Compiler, before execution	Program, during execution
Achieved using	Function overloading, operator overloading	Function overriding, virtual functions
Inheritance needed	No	Yes
Speed	Faster	Slightly slower (virtual table lookup)
Flexibility	Less flexible	More flexible

Q2. What is a virtual function? Why is it needed?

Answer: A virtual function is a member function of a base class declared with the keyword **virtual**, which a derived class can override. It tells the compiler not to use static linkage for the call. Instead, the function to be executed is decided at runtime based on the type of the object being pointed to (dynamic

linkage or late binding). Without virtual, a base-class pointer would always call the base-class version, even if it points to a derived-class object. Virtual functions are therefore needed to achieve runtime polymorphism.

Q3. What is the difference between function overloading and function overriding?

Answer: Function overloading means having multiple functions with the same name but different parameter lists in the same class; it is resolved at compile time. Function overriding means redefining a base-class function (with the same signature) in a derived class; when the base function is virtual, it is resolved at runtime. Overloading needs no inheritance, whereas overriding requires it.

14. Student Declaration

I hereby declare that the above practical work has been completed by me and that the implementation, analysis, and answers are my own work.

Student Name	PRN	Signature
Pratik Akrur Pandit	202501110116	